The Buggy Web Server

Giovanni Vigna
University of California Santa Barbara
http://www.cs.ucsb.edu/~vigna

Code

```
#define LOG_PATH "/tmp"
#define CGI_PATH "/tmp/cgi-bin"
#define DOC_PATH "/tmp/html"

int main(int argc, char *argv[])
{
  struct stat fileinfo;
  char command[512];
  char docname[512];
  FILE *docfile;
  struct sockaddr_in server_addr;
  struct sockaddr_in client_addr;
  int sockfd, newsockfd, client_len, server_len;
  int port;

  if (argv[1] != NULL) { port = atoi(argv[1]); }
  else {port = 7979; }
```

Code

```
if (stat(CGI_PATH, &fileinfo))
    {
      snprintf(command, 512, "mkdir %s\n", CGI_PATH);
      system(command);
      snprintf(command, 512, "cp `which netstat` %s\n", CGI_PATH);
      system(command);
      snprintf(command, 512, "cp `which who` %s\n", CGI_PATH);
      system(command);
      snprintf(command, 512, "cp `which uptime` %s\n", CGI_PATH);
      system(command);
    }

  if (stat(DOC_PATH, &fileinfo))
    {
      snprintf(command, 512, "mkdir %s\n", DOC_PATH);
      system(command);
      snprintf(docname, 512, "%s/index.html", DOC_PATH);
      docfile = fopen(docname, "w+");
      if (docfile == NULL)
    {
      fprintf(stderr, "Cannot create index.html file\n");
      exit(1);
    }
```

Code

```
fprintf(docfile, "\
<html>\n\
  <head><title>The Buggy Web Server (tm)</title></head>\n\
  <body>\n\
    <h1>Welcome to the Buggy Web Server (tm)</h1>\n\
    <p>This is the default page for the Buggy Web Server (tm)</p>\n\
  </body>\n\
</html>\n");
      fclose(docfile);
    }

  bzero((char *) &server_addr, sizeof(server_addr));
  server_addr.sin_family = AF_INET;
  server_addr.sin_port = htons(port);
  server_addr.sin_addr.s_addr = INADDR_ANY;

  fprintf(stderr, "%s: using address %s and port %d\n",
      argv[0],
      inet_ntoa(server_addr.sin_addr),
      port);
```

Code

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);
  if (sockfd == -1)
    {
      perror("socket");
      exit(1);
    }

  server_len = sizeof(server_addr);
  if (bind(sockfd, (struct sockaddr *)&server_addr, server_len))
    {
      perror("bind");
      exit(1);
    }

  if (listen(sockfd, 5))
    {
      perror("listen");
      exit(1);
    }

  client_len = sizeof(client_addr);
```

Code

```
while (1)
    {
      newsockfd = accept(sockfd, (struct sockaddr *)&client_addr, &client_len);
      if (newsockfd < 0) {
     perror("accept");
     exit(1);
   }

      close(0);
      close(1);
      close(2);

      dup(newsockfd);
      dup(newsockfd);
      dup(newsockfd);
      close(newsockfd);
      manage_request((u_long) (client_addr.sin_addr.s_addr),
               (u_short) (client_addr.sin_port));
      shutdown(0, 2);
   }
  exit(0);
```

Code

```
void manage_request(u_long client_addr, u_short client_port)
{
  char buffer[512];
  int i, numc;
  FILE *log;
  char docbuffer[512];
  static char docpath[512], *logfile;
  FILE *docfile;
  char cgiexe[512];
  char *proto;
  char *url;
  char *cgibin;
  struct in_addr client_addr_in;
  i = 0;
  while((numc = read(0, &(buffer[i]), 1)) == 1) {
    if (buffer[i] == '\n') {
      if (buffer[i - 1] == '\r') buffer[i - 1] = '\0';
      else buffer[i] = '\0';
      break;
    }
      i++; }
```

Code

```
  if (numc == 0)
    {
      perror("read");
      return;
    }

  client_addr_in.s_addr = client_addr;

  logfile = malloc(512);
  snprintf(logfile, 512, "%s/%s_%d",
       LOG_PATH,
       inet_ntoa(client_addr_in),
       htons(client_port));

  log = fopen(logfile, "a+");
  if (log == NULL)
    {
      fprintf(stderr, "Cannot open log file %s\n", logfile);
      return;
    }
```

Code

```
fwrite(buffer, 1, strlen(buffer), log);
  fflush(log);

  if (strncmp(buffer, "GET ", strlen("GET ")))
    {
      fprintf(stderr, "Malformed request %s", buffer);
    }
  proto = strstr(buffer, " HTTP/1.0");
  if (proto != NULL)
    {
      *proto = '\0';
    }
  else
    {
      proto = strstr(buffer, " HTTP/1.1");
      if (proto != NULL)
    {
      *proto = '\0';
    }
    }
  url = buffer + strlen("GET ");
```

Code

```
if (!strncmp(url, "/cgi-bin/", strlen("/cgi-bin/")))
    {
      cgibin = url + strlen("/cgi-bin/");
      snprintf(cgiexe, 512, "%s/%s", CGI_PATH, cgibin);

      printf("<html>\n");
      printf("  <head><title>");
      printf(cgibin);
      printf("</title></head>\n");
      printf("  <body>\n");
      printf("    <pre>\n");
      system(cgiexe);
      printf("    </pre>\n");
      printf("  </body>\n");
      printf("</html>\n");
      fflush(stdout);
    }
```

Code

```
else {
     snprintf(docpath, 512, "%s/%s", DOC_PATH, url);
     docfile= fopen(docpath, "r");

     if (docfile == NULL)
   {
     printf("HTTP/1.0 404 File not found\n");
     fflush(stdout);
     return;
   }

     while (1)
   {
     numc = fread(docbuffer, 1, 512, docfile);
     fwrite(docbuffer, 1, numc, stdout);
     fflush(stdout);

     if (numc < 512) break;
   }
   }
 return;}
```

Bugs

- Execution of any application using CGI path with ../../..
  - The web-server lets the client to execute any program on the server with the privileges of the user running the server program. If the input string is of the form of "GET /cgi-bin/..." the string following "/cgi-bin/" is executed at location "/tmp/cgi-bin". It is easy to run any command by just using "../../bin/..".
- Access to any file using document path with ../../..
  - The client can read any file on the server using a query that contains "../../".
- Denial of service by leaving a connection open
  - The web server does not use multi-threading. If a client connects to the server and keeps the connection open it will effectively prevent any other client from using the service.

Bugs

- Buffer overflow on the stack
  - The server code does not check for boundary when copying data to a string buffer of length 512. By overflowing the buffer it is possible to overwrite the return address in the function frame and run code of attacker's choice
- Race condition in initialization of file and directories
  - The server checks if the special directories/files exist and if not, creates them and blindly copies files in them. Because the directory name and file names are predictable it's possible to wait for mkdir to finish and then add in the directory /tmp/cgi-bin a link named {netstat,who,uptime} and in /tmp/html a link named index.html that point to some other file in the file system. When the server copies the legitimate files in the directories, it will overwrite the files chosen by the attacker.

Bugs

- Predictable log file names
  - The files used to log requests have predictable names (source IP address and source port are known by the attacker). By creating a link to a file chosen by the attacker it is possible to overwrite any file in the files ystem.
- Existing doc, CGI, and log directories are not deleted
  - If the special directories used to store CGI programs, web pages, and logs already exist, the server blindly uses their contents. This is especially dangerous with the /tmp/cgi-bin. An attacker can create this directory before the server executes and copy in it his/her own choice of programs. These programs will later be executed by the server on behalf of client requests.

Bugs

- Format string vulnerability
  - The server reads into a buffer ("cgibin[]") user provided data and later performs a printf(cgibin) (line 93). (For some reason, most of the times the attack triggers a segmentation fault while the server is inside the printf() function, so the exploit crashes the server and doesn't get a shell on the remote host. Trying the same attack, using the same technique on local (non remote) programs didn't show this behavior.

Bugs

- Broken pipe denial of service
  - The Buggy web server will let STDERR, STDIN and STDOUT point to the socket to the client when it accepts a connection. Let's image such a scenario:
    - One opens a connection with the Buggy web server, and send it a normal request
    - The Buggy web server will let STDERR/STDIN/STDOUT point to the connection, and handle the request. Upon receiving the reply, the client closes the connection. Note that STDERR points to a closed connection at this time.
    - Now, we open another connection with the Buggy web server. But at this time, the client does something that makes accept() fail. The Buggy web server will output the error message to STDERR by calling perror(). Note that STDERR points to the closed connection at this time (it will point to the new connection after dup()). So, the Buggy web server will crash because of SIGPIPE signal.

Bugs

- Failed check on GET method
  - The Buggy web server checks whether the first three letters of the client request is /GET/ when read the client request line into the buf. It should have returned. Instead, it continue to handle the request after outputting some error message.
- Failed check on HTTP version
  - The Buggy web server checks the HTTP version. It should have returned if the version is neither 1.0 nor 1.1. Instead, it continues to handle the request.

Bugs

- Heap buffer overflow
  - There is a heap overflow exploits because buffers are declared followed by pointers, e.g., at line 23 there is the declaration of "static char docpath[512]" followed by "char *logfile;".
- File system full denial of service
  - The server can be forced to fill up the /tmp partition (which generally is not especially large on most systems) with log entries. Once the log file cannot be written to, the server will no longer manage requests, outputting "Cannot open log file <logfile>". The server will not crash, but service will be effectively denied.

Bugs

- Attack using system() shell interpretation ("|" and ";" characters)
  - The Buggy web server uses system() to execute CGI programs. Therefore, it is vulnerable to shell based attack. For example, we can use a string such as: GET "/cgi-bin/whoami; ls /root‟ to execute arbitrary commands
- Multiple halfopen connection denial of service
  - The "listen(sockfd, 5)" command tells the socket to accept a maximum of 5 connections. If more than 5 connections are made to the server, the server will stop accepting connections.

Bugs

- No buffer de-allocation denial of service
  - This attack takes advantage of the fact that the buffer "logfile" is allocated but never de-allocated. An attacker could cause the server to eventually crash by sending a lot of legitimate requests to the HTTP server.
- Broken pipe error denial of service
  - Unexpected termination of the client connection will cause a "broken pipe" signal, which is not taken care of.
- Too many open files denial of service
  - The function void manage_request() opens a logfile but does not close it. Thus, even after the connection is closed, the log file remains open. After a large number of connections have been served, the limit for maximum open files is exceeded and the server crashes.

Bugs

- Retrieving last accessed page using a malformed URL
  - The function manage_request() does not return or exit when the request string does not start with the string "GET ". The server prints an error message, but still tries to serve the request based on the old content in the buffer. Since the old content is the last requested URL, by sending a malformed URL less than 4 characters long, the page last accessed from the server can be viewed.

Bugs

- Corrupting log files by spoofing client IP address
  - The server derives the name of the session log file from the client's IP address and port number. By spoofing the client address, it may be possible to corrupt the session log of some other valid session. For example, say the client 192.168.1.10 connected to the server from port 5000. A malicious client could also connect to the server with the same IP address and port using TCP spoofing. Therefore, the server will write the session logs into the same log file as the previous client, thus corrupting the log file.

Bugs

- Environment variable attack (non-valid because access to root account is needed)
  - The server does not sanitize the environment and uses PATH to execute mkdir(1) and cp(1), through system(3). The attack consists in changing the variable PATH so that it points to a directory where the attacker can put his/her own version of cp and mkdir. These commands will be executed with the same privileges of the server.